



QUICK REFERENCE

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

VCF Command Reference

Quick-reference guide to essential CLI commands for VCF component management, troubleshooting, and daily operations.

CLI COMMANDS

MANAGEMENT

TROUBLESHOOTING

DAILY OPS

VCF 9.0

VMware Cloud Foundation

VCF 9.0.1 Command Reference

PowerShell users: Every bash block in this document has a paste-ready PowerShell sibling block underneath it (where translation is meaningful). For the full substitution table — `curl` → `curl.exe` , `2>/dev/null` → `2>$null` , here-docs → here-strings, `jq` → `ConvertFrom-Json` , etc. — see [POWERSHELL-TRANSLATION-REFERENCE.md](#). Blocks targeting the appliance shell only (`psql`, `esxcli`, `vim-cmd`, `vmon-cli`, `service-control`, `nodetool`, `kubectl`) are flagged "Run on the appliance only" with no PS sibling.

Table of Contents

1. SDDC Manager API
2. SDDC Manager Services & Logs
3. SDDC Manager PostgreSQL Database
4. SDDC Manager SSH & Account Recovery
5. NSX Manager API
6. NSX Admin CLI
7. NSX Node Services
8. vCenter Server Management
9. vCenter / vSphere API
10. ESXi Host — System
11. ESXi Host — Network
12. ESXi Host — Storage
13. ESXi Host — VM Management
14. ESXi Host — Services & Troubleshooting
15. vSAN Operations
16. vSAN Disk Management
17. vmkfstools — Disk Cloning & Management
18. Certificate Operations — OpenSSL
19. Certificate Operations — Keytool
20. NSX Certificate Replacement (Full Workflow)
21. VCF Operations for Logs — Certificate
22. Offline Depot — Server & Bundles
23. SOS Diagnostic Bundle
24. VDT (VMware Depot Tool)
25. ovftool — Manual OVA Deployment
26. Windows Host — DNS & Firewall
27. Diagnostic Python Scripts
28. GRUB Recovery — SDDC Manager

Where commands run in this reference. Each section is scoped to one environment, indicated in section headers and the comment line at the top of each code block (e.g., `# bash on SDDC Manager appliance`). Common environments used here:

- **SDDC Manager API / shell** — curl/Postman from any Linux client OR SSH to the SDDC Manager appliance (default user: `vcf`).
- **ESXi host shell** — SSH to host root.
- **vCenter shell or PowerCLI** — SSH to VCSA root, OR PowerCLI from a workstation.
- **NSX Manager API / shell** — curl with admin auth, OR SSH to NSX admin then `st en`.
- **VCF Installer** — SSH to the Cloud Builder appliance.

When a command lacks an explicit environment header, read the comment at the top of the block — every block starts with a `# bash on <X>` or `# PowerShell on <X>` header. Replace any credential placeholder like `<your-password>` with values from your vault.

1. SDDC Manager API

Get Auth Token

```
# bash on SDDC Manager appliance via SSH (or any Linux client)
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
-H "Content-Type: application/json" \
-d '{"username":"administrator@vsphere.local","password":"<pass>"}' \
| python3 -c "import sys,json;print(json.load(sys.stdin)['accessToken'])")
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# PS 5.1: trust self-signed certs for this session
[Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12

$sddc = 'sddc-manager.lab.local'
$body = @{ username = 'administrator@vsphere.local'; password = '<pass>' } |
ConvertTo-Json
$resp = Invoke-RestMethod -Method Post -Uri "https://$sddc/v1/tokens" `
-ContentType 'application/json' -Body $body
$TOKEN = $resp.accessToken
```

Core Endpoints

```
# bash on SDDC Manager appliance via SSH
# System health
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/system/health

# NSX cluster status (check for ACTIVE vs ACTIVATING)
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/nsxt-clusters |
python3 -m json.tool

# Resource locks (stale locks block all operations)
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/resource-locks |
python3 -m json.tool

# Credentials
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/credentials |
python3 -m json.tool

# Task details (UI hides error payloads)
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/tasks/<task-id> |
python3 -m json.tool

# System notifications
curl -sk -H "Authorization: Bearer $TOKEN"
https://localhost/v1/system/notifications

# Domains
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/domains | python3
-m json.tool

# Hosts
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/hosts | python3 -
m json.tool

# Failed tasks (newest first)
curl -sk -H "Authorization: Bearer $TOKEN" "https://localhost/v1/tasks?
status=FAILED&pageSize=5&sortOrder=DESC" | python3 -m json.tool

# Stuck IN_PROGRESS tasks (these block new operations)
curl -sk -H "Authorization: Bearer $TOKEN" "https://localhost/v1/tasks?
status=IN_PROGRESS" | python3 -c \
    "import sys,json; d=json.load(sys.stdin); print(f'Count:
{len(d.get(\"elements\",[]))}'); [print(f'  {t[\"type\"]} |
{t[\"creationTimestamp\"]}') for t in d.get('elements',[])]"
```

```
# NSX credentials
curl -sk -H "Authorization: Bearer $TOKEN" "https://localhost/v1/credentials?
resourceType=NSXT_MANAGER" | python3 -m json.tool

# Cancel stuck task (NOTE: API often rejects with TA_TASK_CAN_NOT_BE_RETRIED – use
DB fix instead)
curl -sk -X PATCH https://localhost/v1/tasks/<task-id> \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"status":"CANCELLED"}'
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Requires $TOKEN and $sddc from the auth-token block above
[Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
$h = @{ Authorization = "Bearer $TOKEN" }

# System health
Invoke-RestMethod -Uri "https://$sddc/v1/system/health" -Headers $h

# NSX cluster status (check for ACTIVE vs ACTIVATING)
Invoke-RestMethod -Uri "https://$sddc/v1/nsxt-clusters" -Headers $h | ConvertTo-
Json -Depth 10

# Resource locks (stale locks block all operations)
Invoke-RestMethod -Uri "https://$sddc/v1/resource-locks" -Headers $h | ConvertTo-
Json -Depth 10

# Credentials
Invoke-RestMethod -Uri "https://$sddc/v1/credentials" -Headers $h | ConvertTo-
Json -Depth 10

# Task details (UI hides error payloads)
$taskId = '<task-id>'
Invoke-RestMethod -Uri "https://$sddc/v1/tasks/$taskId" -Headers $h | ConvertTo-
Json -Depth 10

# System notifications
Invoke-RestMethod -Uri "https://$sddc/v1/system/notifications" -Headers $h

# Domains
Invoke-RestMethod -Uri "https://$sddc/v1/domains" -Headers $h | ConvertTo-Json -
```

Depth 10

```
# Hosts
Invoke-RestMethod -Uri "https://$sddc/v1/hosts" -Headers $h | ConvertTo-Json -
Depth 10

# Failed tasks (newest first)
Invoke-RestMethod -Uri "https://$sddc/v1/tasks?
status=FAILED&pageSize=5&sortOrder=DESC" -Headers $h | ConvertTo-Json -Depth 10

# Stuck IN_PROGRESS tasks (these block new operations)
$ip = Invoke-RestMethod -Uri "https://$sddc/v1/tasks?status=IN_PROGRESS" -Headers
$h
"Count: $($ip.elements.Count)"
$ip.elements | ForEach-Object { "  $($_.type) | $($_.creationTimestamp)" }

# NSX credentials
Invoke-RestMethod -Uri "https://$sddc/v1/credentials?resourceType=NSXT_MANAGER" -
Headers $h | ConvertTo-Json -Depth 10

# Cancel stuck task (NOTE: API often rejects with TA_TASK_CAN_NOT_BE_RETRIED – use
DB fix instead)
$cancelBody = @{ status = 'CANCELLED' } | ConvertTo-Json
Invoke-RestMethod -Method Patch -Uri "https://$sddc/v1/tasks/$taskId" `
-Headers $h -ContentType 'application/json' -Body $cancelBody
```

2. SDDC Manager Services & Logs

Service Management

```
# Run on the appliance only (bash on SDDC Manager via SSH)
# Check all VCF services
systemctl status domainmanager lcm operationsmanager commonsvcs postgresql nginx

# Restart all services (recommended method)
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh

# Restart individual services
systemctl restart domainmanager
systemctl restart lcm
```

```
systemctl restart operationsmanager
systemctl restart commonsvcs
systemctl restart sddc-manager-ui-app.service

# List all VCF service units
systemctl list-units --type=service | grep vcf
```

Key Log Locations

```
# Run on the appliance only (bash on SDDC Manager via SSH)
# Domain manager (deployment, bringup)
tail -100 /var/log/vmware/vcf/domainmanager/domainmanager.log

# Operations manager (credential rotation, tasks)
tail -100 /var/log/vmware/vcf/operationsmanager/operationsmanager.log

# Lifecycle manager (bundles, upgrades, certs)
tail -100 /var/log/vmware/vcf/lcm/lcm.log
tail -100 /var/log/vmware/vcf/lcm/lcm-debug.log

# Common services
tail -100 /var/log/vmware/vcf/commonsvcs/commonsvcs.log
```

Log Search

```
# Run on the appliance only (bash on SDDC Manager via SSH)
# Search for errors
grep -ri "error\|exception\|failed"
/var/log/vmware/vcf/domainmanager/domainmanager.log | tail -50

# Search for cert/TLS issues
grep -i "tlsfatal\|ssl\|certificate" /var/log/vmware/vcf/lcm/lcm-debug.log | tail
-20

# Search by timestamp
grep "2026-02-22 14:" /var/log/vmware/vcf/operationsmanager/operationsmanager.log

# Follow log in real time
tail -f /var/log/vmware/vcf/domainmanager/domainmanager.log
```

```
# Search by task ID
grep "<task-id>" /var/log/vmware/vcf/lcm/lcm.log

# Journalctl
journalctl -u vcf-services --since "1 hour ago" -p err
```

Service Properties Ownership Fix

```
# Run on the appliance only (bash on SDDC Manager via SSH)
chown vcf:vcf /opt/vmware/vcf/domainmanager/conf/application-prod.properties
chown vcf:vcf /opt/vmware/vcf/operationsmanager/conf/application-prod.properties
```

NFS Mount Ownership Fix

```
# Run on the appliance only (bash on SDDC Manager via SSH)
chown root:vcf /nfs/vmware/vcf/nfs-mount/
```

3. SDDC Manager PostgreSQL Database

Enable Database Access (Trust Auth Workaround)

```
# Run on the appliance only (psql/pg_ctl on SDDC Manager)
# Backup pg_hba.conf
cp /data/pgdata/pg_hba.conf /data/pgdata/pg_hba.conf.bak

# Set trust auth (temporary – ALWAYS restore after)
sed -i 's/scram-sha-256/trust/g' /data/pgdata/pg_hba.conf
su - postgres -c "/usr/pgsql/15/bin/pg_ctl reload -D /data/pgdata"
```

Connect to Databases

```
# Run on the appliance only (psql on SDDC Manager)
# IMPORTANT: Always disable psql pager to prevent --More-- prompts
export PAGER=cat
export PGPAGER=cat

# Platform database (resource states, locks, task metadata)
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform"

# Operations manager database (tasks, executions)
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d operationsmanager"

# List all databases
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -t -c \"SELECT datname FROM
pg_database WHERE datistemplate = false;\""
# Returns: postgres, lcm, platform, operationsmanager, domainmanager,
sddc_manager_ui

# List tables in a database
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -t -c \"SELECT tablename
FROM pg_tables WHERE schemaname = 'public' ORDER BY tablename;\""

```

Credential Cascade Failure — Full Repair

```
# Run on the appliance only (psql on SDDC Manager)
# — Step 1: Fix NSX resource status (platform DB) —
# Check current status (should be ACTIVE, may be ACTIVATING or ERROR)
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -t -c \"SELECT id,
status FROM nsxt;\""

# Fix ANY non-ACTIVE status
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"UPDATE nsxt SET
status = 'ACTIVE' WHERE status != 'ACTIVE';\""

# — Step 2: Clear stale resource locks (platform DB) —
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"SELECT count(*)
FROM lock;\""
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"DELETE FROM
lock;\""

# — Step 3: Mark stuck tasks as resolved (platform DB) —
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"SELECT resolved,"

```

```
count(*) FROM task_metadata GROUP BY resolved;\\"
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \\"UPDATE
task_metadata SET resolved = true WHERE resolved = false;\\"

# — Step 4: Clear task locks (platform DB) —
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \\"DELETE FROM
task_lock;\\""
```

Key tables: *platform* DB: *nsxt* (status), *lock*, *task_metadata* (resolved), *task_lock*.
operationsmanager DB: *task* (column: *state*, not *status*), *execution* (column:
execution_status), *processing_task*, *execution_to_task*.

Restore pg_hba.conf (CRITICAL — Do Immediately)

```
# Run on the appliance only (psql/pg_ctl on SDDC Manager)
cp /data/pgdata/pg_hba.conf.bak /data/pgdata/pg_hba.conf
su - postgres -c "/usr/pgsql/15/bin/pg_ctl reload -D /data/pgdata"

# Verify restored (should return 4+)
grep -c 'scram-sha-256' /data/pgdata/pg_hba.conf
```

Restart After DB Fix

```
# Run on the appliance only (bash on SDDC Manager via SSH)
systemctl restart operationsmanager
systemctl is-active operationsmanager
```

4. SDDC Manager SSH & Account Recovery

SSH Access

```
# bash from any client (Linux/macOS) using OpenSSH
# Only vcf user can SSH (root/admin rejected)
```

```
ssh vcf@192.168.1.241

# Escalate to root
su -

# File transfer (SCP doesn't work – restricted shell)
ssh vcf@192.168.1.241 "cat > /home/vcf/myfile.zip" < localfile.zip
ssh vcf@192.168.1.241 "cat /path/to/remote/file" > local_copy
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation (Win10/11 OpenSSH client built-in)
# Only vcf user can SSH (root/admin rejected)
ssh vcf@192.168.1.241

# Escalate to root once connected
# su -

# File transfer (SCP doesn't work – restricted shell)
# Push a local file to the appliance via cat redirect:
Get-Content -Raw -Encoding Byte 'localfile.zip' | ssh vcf@192.168.1.241 'cat >
/home/vcf/myfile.zip'
# Pull a remote file to local:
ssh vcf@192.168.1.241 'cat /path/to/remote/file' | Set-Content -Encoding Byte
'local_copy'

# If you have plink (PuTTY) instead, the verified homelab pattern is:
# plink -ssh vcf@192.168.1.241 -pw '<pass>' -m C:\path\to\script.sh
```

Account Lockout (faillock)

```
# Run on the appliance only (faillock is a Linux PAM tool)
# Check lockout status
faillock --user vcf

# Reset locked accounts (run from console as root)
faillock --user vcf --reset
faillock --user root --reset
```

5. NSX Manager API

Cluster & Node Status

```
# bash from any client with curl (or NSX Manager via SSH)
# Cluster health (via VIP)
curl -sk -u admin:'<pass>' https://192.168.1.70/api/v1/cluster/status

# Cluster health (via node – use when VIP is down)
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/cluster/status

# Cluster info + node UUIDs
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/cluster
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12

$nsxVip = '192.168.1.70'
$nsxNode = '192.168.1.71'
$cred = New-Object System.Management.Automation.PSCredential('admin', `
    (ConvertTo-SecureString '<pass>' -AsPlainText -Force))

# Cluster health (via VIP)
Invoke-RestMethod -Uri "https://$nsxVip/api/v1/cluster/status" -Credential $cred

# Cluster health (via node – use when VIP is down)
Invoke-RestMethod -Uri "https://$nsxNode/api/v1/cluster/status" -Credential $cred

# Cluster info + node UUIDs
Invoke-RestMethod -Uri "https://$nsxNode/api/v1/cluster" -Credential $cred
```

Transport & Networking

```
# bash from any client with curl
# Transport node status
```

```

curl -sk -u admin:'<pass>' https://192.168.1.70/api/v1/transport-nodes/state

# Transport zones
curl -sk -u admin:'<pass>' https://192.168.1.70/api/v1/transport-zones

# Segments
curl -sk -u admin:'<pass>' https://192.168.1.70/policy/api/v1/infra/segments

# Compute managers
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/fabric/compute-managers

```

PowerShell equivalent (run on your Windows workstation):

```

# PowerShell on Windows workstation
# Reuses $nsxVip, $nsxNode, $cred from the auth block above
Invoke-RestMethod -Uri "https://$nsxVip/api/v1/transport-nodes/state" -
Credential $cred
Invoke-RestMethod -Uri "https://$nsxVip/api/v1/transport-zones" -
Credential $cred
Invoke-RestMethod -Uri "https://$nsxVip/policy/api/v1/infra/segments" -
Credential $cred
Invoke-RestMethod -Uri "https://$nsxNode/api/v1/fabric/compute-managers" -
Credential $cred

```

Service Health

```

# bash from any client with curl
# Individual service status
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/node/services/http/status
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/node/services/search/status

# Node system status (includes load average)
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/node/status

```

PowerShell equivalent (run on your Windows workstation):

```

# PowerShell on Windows workstation
# Reuses $nsxNode, $cred from the auth block above
Invoke-RestMethod -Uri "https://$nsxNode/api/v1/node/services/http/status" -

```

```
Credential $cred
Invoke-RestMethod -Uri "https://$nsxNode/api/v1/node/services/search/status" -
Credential $cred
Invoke-RestMethod -Uri "https://$nsxNode/api/v1/node/status" -
Credential $cred
```

Certificate Management

```
# bash from any client with curl
# List all certificates
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/trust-
management/certificates

# Import certificate (see Section 20 for full workflow)
curl -sk -u admin:'<pass>' -X POST \
  "https://192.168.1.71/api/v1/trust-management/certificates?action=import" \
  -H "Content-Type: application/json" -d @/tmp/nsx-import.json
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Reuses $nsxNode, $cred from the auth block above
# List all certificates
Invoke-RestMethod -Uri "https://$nsxNode/api/v1/trust-management/certificates" -
Credential $cred

# Import certificate (see Section 20 for full workflow). Payload file built via
Section 20 step 4.
$json = Get-Content -Raw 'C:\temp\nsx-import.json'
Invoke-RestMethod -Method Post -Credential $cred `
  -Uri "https://$nsxNode/api/v1/trust-management/certificates?action=import" `
  -ContentType 'application/json' -Body $json
```

6. NSX Admin CLI

SSH as `admin` user (not root):

```

get cluster status          # Cluster health
get managers               # List manager nodes
get name-servers           # DNS config
set name-servers <ip>      # Set DNS (NOT via UI)
get ntp-servers            # NTP config
set ntp-servers <ip>      # Set NTP (NOT via UI)

```

7. NSX Node Services

SSH as `root` user:

```

# Run on the appliance only (bash on NSX Manager via SSH as root)
# Service status
systemctl status proton          # Main NSX Manager API
systemctl status search         # Search service
systemctl status nsx-appl-proxy # Appliance proxy
systemctl status nsx-sha        # System health agent
systemctl status corfu_server   # Distributed datastore

# Service restart
systemctl restart proton
systemctl restart search
systemctl restart nsx-appl-proxy

# Load monitoring (nested: load >30 = still booting)
cat /proc/loadavg
ps -eo pid,%cpu,%mem,comm --sort=-%cpu | head -15

# Disk space
df -h

```

8. vCenter Server Management

Service Control

```
# Run on the appliance only (bash on vCenter VCSA via SSH)
# Status
service-control --status          # All services summary
service-control --status --all    # All services detailed
service-control --status vpxd     # Specific service

# Start / Stop / Restart
service-control --start --all
service-control --stop vpxd
service-control --restart vpxd
service-control --restart --all

# Alternative service listing
vmon-cli --list
vmon-cli --status vpxd
```

Certificate Manager

```
# Run on the appliance only (bash on vCenter VCSA via SSH)
/usr/lib/vmware-vmca/bin/certificate-manager

# List certificate stores
for store in MACHINE_SSL_CERT TRUSTED_ROOTS; do
    /usr/lib/vmware-vmafd/bin/vecs-cli entry list --store $store
done
```

Database

```
# Run on the appliance only (psql on vCenter VCSA via SSH)
# Check PostgreSQL
service-control --status vmware-vpostgres

# Connect
/opt/vmware/vpostgres/current/bin/psql -U postgres

# Active connections
```

```
/opt/vmware/vpostgres/current/bin/psql -U postgres -c "SELECT count(*) FROM
pg_stat_activity;"
```

Log Locations

```
# Run on the appliance only (bash on vCenter VCSA via SSH)
# VPXD (main vCenter service)
tail -100 /var/log/vmware/vpxd/vpxd.log
grep -i "error\|exception" /var/log/vmware/vpxd/vpxd.log | tail -50

# UI
tail -100 /var/log/vmware/vsphere-ui/logs/vsphere_client_virgo.log

# First boot (during deployment)
cat /var/log/firstboot/firstbootStatus.json
tail -50 /var/log/vmware/firstboot/installer.log
```

Appliance Info

```
# Run on the appliance only (bash on vCenter VCSA via SSH)
vamicli version --appliance
vamicli update --check
```

9. vCenter / vSphere API

```
# bash from any client with curl
# Create session
SESSION=$(curl -sk -X POST https://vcenter.lab.local/api/session \
  -u 'administrator@vsphere.local:<pass>' | tr -d '"')

# List VMs
curl -sk -H "vmware-api-session-id: $SESSION"
https://vcenter.lab.local/api/vcenter/vm
```

```
# MOB (browser-based deep object access)
# https://vcenter.lab.local/mob
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12

$vc = 'vcenter.lab.local'
$cred = New-Object
System.Management.Automation.PSCredential('administrator@vsphere.local', `
    (ConvertTo-SecureString '<pass>' -AsPlainText -Force))

# Create session (response body is a quoted string – strip quotes)
$SESSION = (Invoke-RestMethod -Method Post -Uri "https://$vc/api/session" -
    Credential $cred).ToString().Trim('"')

# List VMs
Invoke-RestMethod -Uri "https://$vc/api/vcenter/vm" -Headers @{ 'vmware-api-
    session-id' = $SESSION }

# MOB (browser-based deep object access)
# Start-Process "https://$vc/mob"
```

10. ESXi Host — System

```
# Run on the appliance only (esxcli on ESXi host shell)
esxcli system hostname get # Show FQDN
esxcli system hostname set --fqdn=esxi01.lab.local # Set FQDN
esxcli system version get # ESXi version/build
esxcli system time get # System time
esxcli system maintenanceMode get # Check maintenance mode
esxcli system maintenanceMode set -e true -m noAction # Enter (no data
    evacuation)
esxcli system maintenanceMode set -e true -m evacuateAllData # Enter (vSAN
    evacuate)
esxcli system maintenanceMode set -e false # Exit maintenance mode
esxcli system ssh set --enable=true # Enable SSH
```

<code>esxcli system ssh get</code>	# SSH status
<code>esxcli hardware cpu global get</code>	# CPU info (EVC check)

11. ESXi Host — Network

```
# Run on the appliance only (esxcli/vmkping/esxcfg on ESXi host shell)
# VMkernel adapters
esxcli network ip interface list
esxcli network ip interface ipv4 get -i vmk0
esxcli network ip interface add -i vmk1 -p "vMotion"
esxcli network ip interface remove -i vmk1

# Routing
esxcli network ip route ipv4 list
esxcli network ip neighbor list

# vSwitch
esxcli network vswitch standard list
esxcli network vswitch dvs vmware list

# Physical NICs
esxcli network nic list
esxcli network nic get -n vmnic0
esxcli network nic stats get -n vmnic0

# Connections / Firewall
esxcli network ip connection list
esxcli network ip connection list | grep 1234
esxcli network firewall ruleset list
esxcli network firewall ruleset list | grep -i ssh

# vmkping
vmkping 192.168.1.75
vmkping -I vmk2 192.168.1.75
vmkping -d -s 1572 192.168.1.75

# Basic ping
# Ping from specific vmk
# Jumbo frame test

# esxcfg
esxcfg-vmknics -l

# VMkernel NIC list
```

```
esxcfg-vswitch -l          # vSwitch list
esxcfg-nics -l             # Physical NIC list
```

12. ESXi Host — Storage

```
# Run on the appliance only (esxcli/vdq/partedUtil on ESXi host shell)
# Device listing
esxcli storage core device list
esxcli storage core device list | grep -E "Display Name|Is SSD"
esxcli storage core adapter list
esxcli storage filesystem list
esxcli storage vmfs extent list

# Rescan
esxcli storage core adapter rescan --all

# SSD detection (force for nested)
esxcli storage nmp satp rule add -s VMW_SATP_LOCAL -d <device> -o enable_ssd
esxcli storage core claiming reclaim -d <device>
esxcli storage nmp satp rule list | grep enable_ssd

# Disk eligibility for vSAN
vdq -qH
vdq -q -d <device>

# Partition management
partedUtil getptbl /vmfs/devices/disks/<device>
partedUtil delete /vmfs/devices/disks/<device> 1
partedUtil delete /vmfs/devices/disks/<device> 2
```

13. ESXi Host — VM Management

```
# Run on the appliance only (vim-cmd on ESXi host shell)
# List VMs
vim-cmd vmsvc/getallvms
```

```

# Power operations
vim-cmd vmshvc/power.getstate <vmid>
vim-cmd vmshvc/power.on <vmid>
vim-cmd vmshvc/power.off <vmid>
vim-cmd vmshvc/power.shutdown <vmid>
vim-cmd vmshvc/power.reset <vmid>

# Register / Unregister
vim-cmd solo/registervm "/vmfs/volumes/<datastore>/<vm>/<vm>.vmx"
vim-cmd vmshvc/unregister <vmid>

# Device info
vim-cmd vmshvc/device.getdevices <vmid>

# BIOS boot
vim-cmd vmshvc/setboot.options <vmid> enterBIOSSetup=true

# Hardware info
vim-cmd hostshvc/hosthardware

# Maintenance mode
vim-cmd hostshvc/maintenance_mode_enter
vim-cmd hostshvc/maintenance_mode_exit

```

14. ESXi Host — Services & Troubleshooting

Service Management

```

# Run on the appliance only (bash on ESXi host shell via SSH)
# Host services
/etc/init.d/hostd restart
/etc/init.d/hostd status
/etc/init.d/vpxa restart
/etc/init.d/vpxa status
services.sh restart # Restart all host services

# SSH
/etc/init.d/SSH start
/etc/init.d/SSH stop
/etc/init.d/SSH status

```

```
vim-cmd hostsvc/enable_ssh && vim-cmd hostsvc/start_ssh
```

```
# NSX services on host
/etc/init.d/nsx-proxy status
/etc/init.d/nsx-proxy restart
/etc/init.d/nsx-datapath status
/etc/init.d/nsx-opsagent status

# Auto-backup
/sbin/auto-backup.sh
```

Certificate Regeneration

```
# Run on the appliance only (bash on ESXi host shell via SSH)
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak
/sbin/generate-certificates
services.sh restart
openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout | grep -A1 "Subject
Alternative Name"
```

Get Host Thumbprints

```
# bash from any client with openssl
echo | openssl s_client -connect 192.168.1.74:443 2>/dev/null | openssl x509 -
noout -fingerprint -sha256
echo | openssl s_client -connect 192.168.1.75:443 2>/dev/null | openssl x509 -
noout -fingerprint -sha256
echo | openssl s_client -connect 192.168.1.76:443 2>/dev/null | openssl x509 -
noout -fingerprint -sha256
echo | openssl s_client -connect 192.168.1.82:443 2>/dev/null | openssl x509 -
noout -fingerprint -sha256
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Pure .NET – no openssl required. Returns SHA-256 thumbprint of the cert
presented on :443.
```

```
function Get-RemoteCertSha256 {
    param([string]$HostName, [int]$Port = 443)
    $tcp = New-Object System.Net.Sockets.TcpClient($HostName, $Port)
    try {
        $ssl = New-Object System.Net.Security.SslStream($tcp.GetStream(), $false,
({$true}))
        $ssl.AuthenticateAsClient($HostName)
        $cert = New-Object
System.Security.Cryptography.X509Certificates.X509Certificate2
$ssl.RemoteCertificate
        $sha = [System.Security.Cryptography.SHA256]::Create()
        $hash = $sha.ComputeHash($cert.RawData)
        ($hash | ForEach-Object { $_.ToString('X2') }) -join ':'
    } finally { $tcp.Close() }
}

'192.168.1.74','192.168.1.75','192.168.1.76','192.168.1.82' | ForEach-Object {
    "$_ : $(Get-RemoteCertSha256 $_)"
}
```

Performance / Diagnostics

```
# Run on the appliance only (esxtop on ESXi host shell)
esxtop                                     # Interactive performance monitor
esxtop -b -d 5 -n 10 > /tmp/esxtop.csv    # Batch mode to CSV
```

vMotion Fix (vhv.enable Ghost Setting)

```
# Run on the appliance only (bash on ESXi host shell via SSH; touches .vmx in
/vmfs)
# Diagnose
grep -i vhv /vmfs/volumes/<datastore>/<vm>/<vm>.vmx

# Fix (removing the line is NOT enough – must set explicitly)
echo 'vhv.enable = "FALSE"' >> /vmfs/volumes/<datastore>/<vm>/<vm>.vmx
```

NSX Host Logs

```
# Run on the appliance only (bash on ESXi host shell via SSH)
tail -50 /var/log/nsx-syslog.log
esxcli network ip connection list | grep 1234
esxcfg-vmknic -l | grep -i tep
```

15. vSAN Operations

Health & Status

```
# Run on the appliance only (esxcli on ESXi host shell)
esxcli vsan health cluster list           # Health summary
esxcli vsan health cluster get           # Detailed health
esxcli vsan health cluster get -t "Network health"
esxcli vsan cluster get                   # Cluster info
esxcli vsan network list                  # vSAN network config
esxcli vsan debug resync summary get      # Resync status
esxcli vsan debug object health summary get # Object health
esxcli vsan debug object list             # All objects
```

Storage & Disk Groups

```
# Run on the appliance only (esxcli on ESXi host shell)
esxcli vsan storage list                  # Disk/storage status
esxcli vsan storage automode set --enabled=false
esxcli vsan storage automode set --enabled=true
esxcli vsan storage add -s <cache> -d <capacity>
esxcli vsan storage remove -s <device>
```

Network

```
# Run on the appliance only (esxcli/vmkping on ESXi host shell)
esxcli vsan network ip add -i vmk1
```

```
esxcli vsan network ip remove -i vmk1
vmkping -I vmk2 192.168.12.120 # TEP-to-TEP connectivity
```

Logging

```
# Run on the appliance only (bash on ESXi host shell via SSH)
grep -i "vsan\|cmmnds\|clom\|dom\|lsom" /var/log/vmkernel.log | tail -50
tail -50 /var/log/vsanmgmt.log
tail -50 /var/log/vsantraced.log
/etc/init.d/vsanmgmt status
/etc/init.d/vsand status
```

vSAN Observer (RVC)

```
# Run on the appliance only (rvc on vCenter VCSA via SSH)
rvc administrator@vsphere.local@localhost
cd /vcenter.lab.local/vcenter-dc01/computers/vcenter-cl01
vsan.observer . --run-webserver --force
```

16. vSAN Disk Management

Remove Disk from vSAN & Clean Partitions

```
# Run on the appliance only (esxcli/partedUtil/vdq on ESXi host shell)
esxcli vsan storage remove -d <device>
partedUtil getptbl /vmfs/devices/disks/<device>
partedUtil delete /vmfs/devices/disks/<device> 1
partedUtil delete /vmfs/devices/disks/<device> 2
vdq -q # Verify eligible again
```

17. vmkfstools — Disk Cloning & Management

```
# Run on the appliance only (vmkfstools on ESXi host shell)
# Clone thick to thin
vmkfstools -i <source>.vmdk <dest>.vmdk -d thin

# Clone thick to thick
vmkfstools -i <source>.vmdk <dest>.vmdk -d zeroedthick
vmkfstools -i <source>.vmdk <dest>.vmdk -d eagerzeroedthick

# Create new disk
vmkfstools -c 50G -d thin /vmfs/volumes/<ds>/<vm>/newdisk.vmdk

# Extend disk
vmkfstools -X 100G /vmfs/volumes/<ds>/<vm>/disk.vmdk

# Delete disk
vmkfstools -U /vmfs/volumes/<ds>/<vm>/disk.vmdk

# Get disk geometry
vmkfstools -g /vmfs/volumes/<ds>/<vm>/disk.vmdk

# Disk descriptor info
vmkfstools -D "/vmfs/volumes/<ds>/<vm>/<vm>.vmdk"
```

18. Certificate Operations — OpenSSL

Generate Self-Signed Certificate

```
# bash on appliance via SSH (or any client with openssl)
openssl req -x509 -newkey rsa:2048 -keyout server.key -out server.crt \
  -days 365 -nodes -subj "/CN=hostname"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Native PS – no openssl required. Exports cert (.crt PEM-style) and PEM-encoded
private key.
$cert = New-SelfSignedCertificate -DnsName 'hostname' -CertStoreLocation
```

```
'Cert:\CurrentUser\My' `
    -NotAfter (Get-Date).AddDays(365) -KeyExportPolicy Exportable -
KeyAlgorithm RSA -KeyLength 2048
# Export public cert (PEM)
"-----BEGIN CERTIFICATE-----
`n$([Convert]::ToBase64String($cert.RawData,'InsertLineBreaks'))`n-----END
CERTIFICATE-----" | Out-File -Encoding ascii server.crt
# Export private key (PEM, PKCS#8)
$rsa =
[System.Security.Cryptography.X509Certificates.RSACertificateExtensions]::GetRSAPriv
"-----BEGIN PRIVATE KEY-----
`n$([Convert]::ToBase64String($rsa.ExportPkcs8PrivateKey(),'InsertLineBreaks'))`n-
-----END PRIVATE KEY-----" | Out-File -Encoding ascii server.key
# If openssl.exe is on PATH (e.g., from Git for Windows), the bash command works
verbatim:
# & openssl.exe req -x509 -newkey rsa:2048 -keyout server.key -out server.crt -
days 365 -nodes -subj "/CN=hostname"
```

Generate with SANs

```
# bash on appliance via SSH (or any client with openssl)
openssl req -x509 -newkey rsa:2048 -keyout server.key -out server.crt \
    -days 365 -nodes -subj "/CN=192.168.1.52/O=VCF-Depot/C=US" \
    -addext "subjectAltName=IP:192.168.1.52,DNS:localhost" \
    -addext "keyUsage=digitalSignature,keyEncipherment" \
    -addext "extendedKeyUsage=serverAuth"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Native PS with SANs. -DnsName accepts both DNS and IP entries.
$cert = New-SelfSignedCertificate `
    -Subject 'CN=192.168.1.52, O=VCF-Depot, C=US' `
    -DnsName '192.168.1.52','localhost' `
    -CertStoreLocation 'Cert:\CurrentUser\My' `
    -NotAfter (Get-Date).AddDays(365) `
    -KeyExportPolicy Exportable -KeyAlgorithm RSA -KeyLength 2048 `
    -KeyUsage DigitalSignature, KeyEncipherment `
    -TextExtension @('2.5.29.37={text}1.3.6.1.5.5.7.3.1') # extendedKeyUsage =
serverAuth
# Export PEM cert + key
```

```
"-----BEGIN CERTIFICATE-----
`n$([Convert]::ToBase64String($cert.RawData,'InsertLineBreaks'))`n-----END
CERTIFICATE-----" | Out-File -Encoding ascii server.crt
$rsa =
[System.Security.Cryptography.X509Certificates.RSACertificateExtensions]::GetRSAPriv
"-----BEGIN PRIVATE KEY-----
`n$([Convert]::ToBase64String($rsa.ExportPkcs8PrivateKey(),'InsertLineBreaks'))`n-
-----END PRIVATE KEY-----" | Out-File -Encoding ascii server.key
# Or: if openssl.exe on PATH, run the bash one-liner verbatim with `& openssl.exe
...`.
```

Generate CSR

```
# bash on appliance via SSH (or any client with openssl)
openssl genrsa -out server.key 2048
openssl req -new -key server.key -out server.csr -subj "/CN=hostname/O=Org/C=US"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Easiest path: shell out to openssl.exe (Git for Windows provides it).
& openssl.exe genrsa -out server.key 2048
& openssl.exe req -new -key server.key -out server.csr -subj
"/CN=hostname/O=Org/C=US"

# Pure-PS alternative using certreq.exe (built into Windows):
@"
[NewRequest]
Subject = "CN=hostname, O=Org, C=US"
KeyLength = 2048
KeyAlgorithm = RSA
MachineKeySet = false
RequestType = PKCS10
Exportable = TRUE
"@ | Out-File -Encoding ascii server.inf
certreq.exe -new server.inf server.csr
```

View Certificate Details

```
# bash on appliance via SSH (or any client with openssl)
openssl x509 -in cert.crt -text -noout # Full details
openssl x509 -in cert.crt -noout -subject # Subject only
openssl x509 -in cert.crt -noout -issuer # Issuer only
openssl x509 -in cert.crt -noout -dates # Valid dates
openssl x509 -in cert.crt -noout -enddate # Expiry only
openssl x509 -in cert.crt -text -noout | grep -A2 "Subject Alternative" # SANs
openssl x509 -in cert.crt -noout -fingerprint -sha256 # Fingerprint
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Pure .NET – no openssl required.
$c = New-Object System.Security.Cryptography.X509Certificates.X509Certificate2
'cert.crt'

$c | Format-List * # Full details
$c.Subject # Subject only
$c.Issuer # Issuer only
"NotBefore: $($c.NotBefore) NotAfter: $($c.NotAfter)" # Valid dates
$c.NotAfter # Expiry only
($c.Extensions | Where-Object { $_.OID.Value -eq '2.5.29.17' }).Format($true) #
SANs
$c.GetCertHashString('SHA256') -replace '(\.)', '$1:' -replace ':$', '' #
Fingerprint
```

Check Remote Certificate

```
# bash from any client with openssl
# View remote cert
echo | openssl s_client -connect <host>:443 2>/dev/null | openssl x509 -noout -
text | grep -A2 "Subject Alternative"

# Download remote cert to file
openssl s_client -showcerts -connect <host>:443 < /dev/null 2>/dev/null | openssl
x509 -outform PEM > /tmp/cert.pem

# Test TLS connectivity
openssl s_client -connect <host>:<port>
```

```
openssl s_client -connect <host>:<port> -tls1_2
openssl s_client -connect <host>:<port> -tls1_2 </dev/null 2>&1 | grep -E
"Cipher|Protocol|Verify"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Pure .NET – no openssl required.
function Get-RemoteCert {
    param([string]$HostName, [int]$Port = 443)
    $tcp = New-Object System.Net.Sockets.TcpClient($HostName, $Port)
    try {
        $ssl = New-Object System.Net.Security.SslStream($tcp.GetStream(), $false,
        ({ $true } ))
        $ssl.AuthenticateAsClient($HostName)
        New-Object System.Security.Cryptography.X509Certificates.X509Certificate2
        $ssl.RemoteCertificate
    } finally { $tcp.Close() }
}

$target = '<host>'
# View remote cert (SAN extension)
$rc = Get-RemoteCert -HostName $target -Port 443
($rc.Extensions | Where-Object { $_.OID.Value -eq '2.5.29.17' }).Format($true)

# Download remote cert to PEM file
$pem = "-----BEGIN CERTIFICATE-----`n" +
    [Convert]::ToBase64String($rc.RawData, 'InsertLineBreaks') +
    "`n-----END CERTIFICATE-----"
$pem | Out-File -Encoding ascii C:\temp\cert.pem

# Test TLS connectivity (negotiated protocol + cipher)
$tcp = New-Object System.Net.Sockets.TcpClient($target, 443)
$ssl = New-Object System.Net.Security.SslStream($tcp.GetStream(), $false,
        ({ $true } ))
$ssl.AuthenticateAsClient($target, $null, [Net.SecurityProtocolType]::Tls12,
        $false)
"Protocol: $($ssl.SslProtocol) Cipher:
$($ssl.CipherAlgorithm)/$($ssl.CipherStrength) Verified:
$($ssl.IsAuthenticated)"
$tcp.Close()
```

Convert Formats

```
# bash on appliance via SSH (or any client with openssl)
openssl x509 -in cert.pem -outform der -out cert.der          # PEM → DER
openssl x509 -in cert.der -inform der -outform pem -out cert.pem # DER → PEM
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# PEM -> DER
$c = New-Object System.Security.Cryptography.X509Certificates.X509Certificate2
'cert.pem'
[IO.File]::WriteAllBytes('cert.der',
$c.Export([System.Security.Cryptography.X509Certificates.X509ContentType]::Cert))

# DER -> PEM
$c = New-Object System.Security.Cryptography.X509Certificates.X509Certificate2
'cert.der'
"-----BEGIN CERTIFICATE-----
`n$([Convert]::ToBase64String($c.RawData,'InsertLineBreaks'))`n-----END
CERTIFICATE-----" |
    Out-File -Encoding ascii cert.pem
```

Verify Certificate

```
# bash on appliance via SSH (or any client with openssl)
openssl verify -CAfile ca.crt server.crt
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
$ca = New-Object System.Security.Cryptography.X509Certificates.X509Certificate2
'ca.crt'
$leaf = New-Object System.Security.Cryptography.X509Certificates.X509Certificate2
'server.crt'
$chain = New-Object System.Security.Cryptography.X509Certificates.X509Chain
$chain.ChainPolicy.RevocationMode = 'NoCheck'
```

```
$chain.ChainPolicy.ExtraStore.Add($ca) | Out-Null
$ok = $chain.Build($leaf)
"server.crt: $(if ($ok) { 'OK' } else { 'FAIL' })"
$chain.ChainStatus | Format-Table Status, StatusInformation
```

19. Certificate Operations — Keytool

List Certificates

```
# Run on the appliance only (paths are appliance-side; keystore lives on SDDC
Manager)
keytool -list -keystore /etc/alternatives/jre/lib/security/cacerts -storepass
changeit
keytool -list -alias <alias> -keystore /etc/alternatives/jre/lib/security/cacerts
-storepass changeit -v
```

Import Certificate

```
# Run on the appliance only (keystore paths are appliance-side)
# Into Java cacerts
keytool -importcert -alias <alias> -file /tmp/cert.crt \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit -noprompt

# Into VCF trust store
KEY=$(cat /etc/vmware/vcf/commonsvcs/trusted_certificates.key)
keytool -importcert -alias <alias> -file /tmp/cert.crt \
  -keystore /etc/vmware/vcf/commonsvcs/trusted_certificates.store \
  -storepass "$KEY" -noprompt
```

Delete Certificate

```
# Run on the appliance only (keystore path is appliance-side)
keytool -delete -alias <alias> \
```

```
-keystore /etc/alternatives/jre/lib/security/cacerts -storepass changeit
```

Find All Cacerts on System

```
# Run on the appliance only (find/cacerts on SDDC Manager filesystem)
find / -name "cacerts" -type f 2>/dev/null
```

Restart After Keystore Changes

```
# Run on the appliance only (bash on SDDC Manager via SSH)
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh
```

20. NSX Certificate Replacement (Full Workflow)

Step 1 — Create OpenSSL Config

```
# bash on appliance via SSH (or any client with openssl)
cat > /tmp/nsx-cert.conf << 'EOF'
[ req ]
default_bits = 2048
distinguished_name = req_distinguished_name
req_extensions = req_ext
x509_extensions = req_ext
prompt = no

[ req_distinguished_name ]
countryName = US
stateOrProvinceName = Lab
localityName = Lab
organizationName = lab.local
commonName = nsx-vip.lab.local

[ req_ext ]
basicConstraints = CA:FALSE
subjectAltName = @alt_names
```

```
[alt_names]
DNS.1 = nsx-vip.lab.local
DNS.2 = nsx-node1.lab.local
DNS.3 = nsx-manager.lab.local
IP.1 = 192.168.1.70
IP.2 = 192.168.1.71
EOF
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Here-string with literal-quote (@'...'@) so $-signs aren't expanded.
$cnf = @'
[ req ]
default_bits = 2048
distinguished_name = req_distinguished_name
req_extensions = req_ext
x509_extensions = req_ext
prompt = no

[ req_distinguished_name ]
countryName = US
stateOrProvinceName = Lab
localityName = Lab
organizationName = lab.local
commonName = nsx-vip.lab.local

[ req_ext ]
basicConstraints = CA:FALSE
subjectAltName = @alt_names

[alt_names]
DNS.1 = nsx-vip.lab.local
DNS.2 = nsx-node1.lab.local
DNS.3 = nsx-manager.lab.local
IP.1 = 192.168.1.70
IP.2 = 192.168.1.71
'@
$cnf | Out-File -Encoding ascii C:\temp\nsx-cert.conf
```

Step 2 — Generate Certificate

```
# bash on appliance via SSH (or any client with openssl)
openssl req -x509 -nodes -days 825 -newkey rsa:2048 \
  -keyout /tmp/nsx.key -out /tmp/nsx.crt \
  -config /tmp/nsx-cert.conf -sha256
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Easiest: openssl.exe (Git for Windows ships it). Backslash continuation ->
backtick.
& openssl.exe req -x509 -nodes -days 825 -newkey rsa:2048 `
  -keyout C:\temp\nsx.key -out C:\temp\nsx.crt `
  -config C:\temp\nsx-cert.conf -sha256

# Pure-PS alternative for SAN cert without openssl:
# $cert = New-SelfSignedCertificate -Subject 'CN=nsx-vip.lab.local, O=lab.local,
C=US' `
#   -DnsName 'nsx-vip.lab.local','nsx-node1.lab.local','nsx-
manager.lab.local','192.168.1.70','192.168.1.71' `
#   -CertStoreLocation 'Cert:\CurrentUser\My' -NotAfter (Get-Date).AddDays(825)
#   -KeyExportPolicy Exportable -KeyAlgorithm RSA -KeyLength 2048
```

Step 3 — Verify SANs

```
# bash on appliance via SSH (or any client with openssl)
openssl x509 -in /tmp/nsx.crt -text -noout | grep -A4 "Subject Alternative Name"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
$c = New-Object System.Security.Cryptography.X509Certificates.X509Certificate2
'C:\temp\nsx.crt'
($c.Extensions | Where-Object { $_.Oid.Value -eq '2.5.29.17' }).Format($true)
```

Step 4 — Build JSON Payload

```
# bash on appliance via SSH (or any client with python3)
python -c "
import json
cert = open('/tmp/nsx.crt').read()
key = open('/tmp/nsx.key').read()
print(json.dumps({'pem_encoded': cert, 'private_key': key}))
" > /tmp/nsx-import.json
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
$payload = @{
    pem_encoded = (Get-Content -Raw 'C:\temp\nsx.crt')
    private_key = (Get-Content -Raw 'C:\temp\nsx.key')
} | ConvertTo-Json
$payload | Out-File -Encoding ascii C:\temp\nsx-import.json
```

Step 5 — Import via NSX API

```
# bash from any client with curl
curl -sk -u admin:'<pass>' -X POST \
  "https://192.168.1.71/api/v1/trust-management/certificates?action=import" \
  -H "Content-Type: application/json" -d @/tmp/nsx-import.json
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12

$nsxNode = '192.168.1.71'
$cred = New-Object System.Management.Automation.PSCredential('admin', `
  (ConvertTo-SecureString '<pass>' -AsPlainText -Force))

$json = Get-Content -Raw 'C:\temp\nsx-import.json'
$import = Invoke-RestMethod -Method Post -Credential $cred `
  -Uri "https://$nsxNode/api/v1/trust-management/certificates?action=import" `
```

```
-ContentType 'application/json' -Body $json
$import # capture $import.results[0].id (or .id) for next steps
```

Step 6 — Get Node UUID

```
# bash from any client with curl
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/cluster
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Reuses $nsxNode, $cred from Step 5
Invoke-RestMethod -Uri "https://$nsxNode/api/v1/cluster" -Credential $cred
```

Step 7 — Apply to Node

```
# bash from any client with curl
curl -sk -u admin:'<pass>' -X POST \
  "https://192.168.1.71/api/v1/trust-management/certificates/<cert-id>?
  action=apply_certificate&service_type=API&node_id=<node-uuid>"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
$certId = '<cert-id>'
$nodeId = '<node-uuid>'
Invoke-RestMethod -Method Post -Credential $cred `
  -Uri "https://$nsxNode/api/v1/trust-management/certificates/$certId`?
  action=apply_certificate&service_type=API&node_id=$nodeId"
```

Step 8 — Apply to VIP

```
# bash from any client with curl
curl -sk -u admin:'<pass>' -X POST \
```

```
"https://192.168.1.71/api/v1/trust-management/certificates/<cert-id>?
action=apply_certificate&service_type=MGMT_CLUSTER"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
Invoke-RestMethod -Method Post -Credential $cred `
  -Uri "https://$nsxNode/api/v1/trust-management/certificates/$certId`?
action=apply_certificate&service_type=MGMT_CLUSTER"
```

Step 9 — Verify

```
# bash from any client with openssl
openssl s_client -connect 192.168.1.71:443 -showcerts </dev/null 2>/dev/null |
openssl x509 -noout -text | grep -A2 "Subject Alternative Name"
openssl s_client -connect 192.168.1.70:443 -showcerts </dev/null 2>/dev/null |
openssl x509 -noout -text | grep -A2 "Subject Alternative Name"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
function Get-RemoteCertSAN {
    param([string]$HostName, [int]$Port = 443)
    $tcp = New-Object System.Net.Sockets.TcpClient($HostName, $Port)
    try {
        $ssl = New-Object System.Net.Security.SslStream($tcp.GetStream(), $false,
        ({ $true } ))
        $ssl.AuthenticateAsClient($HostName)
        $cert = New-Object
        System.Security.Cryptography.X509Certificates.X509Certificate2
        $ssl.RemoteCertificate
        ($cert.Extensions | Where-Object { $_.Oid.Value -eq '2.5.29.17'
    }).Format($true)
    } finally { $tcp.Close() }
}
"Node 71: $(Get-RemoteCertSAN '192.168.1.71')"
"VIP 70: $(Get-RemoteCertSAN '192.168.1.70')"
```

Step 10 — Import into SDDC Manager Trust Stores

```
# Run on the appliance only (keytool keystores live on SDDC Manager)
# Pull active NSX cert
openssl s_client -showcerts -connect 192.168.1.71:443 < /dev/null 2>/dev/null |
openssl x509 -outform PEM > /tmp/nsx-root.crt

# VCF trust store
KEY=$(cat /etc/vmware/vcf/commons/vcs/trusted_certificates.key)
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
  -keystore /etc/vmware/vcf/commons/vcs/trusted_certificates.store \
  -storepass "$KEY" -noprompt

# Java cacerts
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit -noprompt

# Restart services
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh
```

21. VCF Operations for Logs — Certificate

Create OpenSSL Config

```
# bash on appliance via SSH (or any client with openssl)
cat > /tmp/vrli-cert.cnf << 'EOF'
[req]
default_bits = 4096
prompt = no
default_md = sha256
distinguished_name = dn
req_extensions = v3_req
x509_extensions = v3_req

[dn]
C = US
ST = California
```

```

L = Lab
O = Lab
OU = VCF
CN = logs.lab.local

[v3_req]
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth, clientAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = logs.lab.local
DNS.2 = logs
IP.1 = 192.168.1.242
EOF

```

PowerShell equivalent (run on your Windows workstation):

```

# PowerShell on Windows workstation
$cnf = @'
[req]
default_bits = 4096
prompt = no
default_md = sha256
distinguished_name = dn
req_extensions = v3_req
x509_extensions = v3_req

[dn]
C = US
ST = California
L = Lab
O = Lab
OU = VCF
CN = logs.lab.local

[v3_req]
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth, clientAuth
subjectAltName = @alt_names

```

```
[alt_names]
DNS.1 = logs.lab.local
DNS.2 = logs
IP.1 = 192.168.1.242
'@
$cnf | Out-File -Encoding ascii C:\temp\vrli-cert.cnf
```

Generate and Verify

```
# bash on appliance via SSH (or any client with openssl)
openssl req -x509 -nodes -days 730 -newkey rsa:4096 \
  -keyout /tmp/vrli.key -out /tmp/vrli.crt -config /tmp/vrli-cert.cnf

openssl x509 -in /tmp/vrli.crt -noout -text | grep -A5 "Subject Alternative Name"
cat /tmp/vrli.crt
cat /tmp/vrli.key
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation (assumes openssl.exe on PATH)
& openssl.exe req -x509 -nodes -days 730 -newkey rsa:4096 `
  -keyout C:\temp\vrli.key -out C:\temp\vrli.crt -config C:\temp\vrli-cert.cnf

# Verify SANs
$c = New-Object System.Security.Cryptography.X509Certificates.X509Certificate2
'C:\temp\vrli.crt'
($c.Extensions | Where-Object { $_.Oid.Value -eq '2.5.29.17' }).Format($true)

# Show PEM contents
Get-Content C:\temp\vrli.crt
Get-Content C:\temp\vrli.key
```

22. Offline Depot — Server & Bundles

Windows — Start HTTPS Depot Server

```
cd C:\VCF-DEPOT
python https_server.py
```

Windows — Generate Certificate

```
cd C:\VCF-DEPOT
python generate_cert.py
```

Windows — Firewall Rule

```
netsh advfirewall firewall add rule name="VCF Depot 8443" dir=in action=allow
protocol=tcp localport=8443
```

Windows — Extract Metadata

```
Expand-Archive -Path "vcf-9.0.1.0-offline-depot-metadata.zip" -DestinationPath
"C:\VCF-Depot\metadata-extract" -Force
Copy-Item "C:\VCF-Depot\metadata-extract\PROD\*" "C:\VCF-Depot\PROD\" -Recurse -
Force
```

SDDC Manager — Import Depot Certificate

```
# Run on the appliance only (cacerts keystore + systemctl live on SDDC Manager)
# Download cert from depot server
openssl s_client -connect 192.168.1.160:8443 </dev/null 2>/dev/null | openssl x509
-outform PEM > /tmp/depot.crt

# Import into Java truststore
CACERTS=$(find /usr -name cacerts 2>/dev/null | head -1)
keytool -importcert -trustcacerts -alias offline-depot -file /tmp/depot.crt \
  -keystore $CACERTS -storepass changeit -noprompt

# Verify
```

```
keytool -list -alias offline-depot -keystore $CACERTS -storepass changeit

# Restart LCM
systemctl restart lcm
```

SDDC Manager — Verify Depot Connectivity

```
# bash on appliance via SSH (or any client with curl that can reach the depot)
curl -k -u admin:admin
https://192.168.1.160:8443/PROD/COMP/SDDC_MANAGER_VCF/lcm/productVersionCatalog/prod
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
[Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12

$depotCred = New-Object System.Management.Automation.PSCredential('admin', `
    (ConvertTo-SecureString 'admin' -AsPlainText -Force))
Invoke-RestMethod -Credential $depotCred `
    -Uri
'https://192.168.1.160:8443/PROD/COMP/SDDC_MANAGER_VCF/lcm/productVersionCatalog/prod'
```

23. SOS Diagnostic Bundle

```
# Run on the appliance only (SOS lives at /opt/vmware/sddc-support/ on SDDC
Manager)
cd /opt/vmware/sddc-support/

# Collect log bundle
./sos --domain-name mgmt --log-bundle

# Log bundle with health check
./sos --domain-name mgmt --log-bundle --health-check

# Health check only
```

```
./sos --health-check

# Get inventory
./sos --get-inventory

# Get all passwords
./sos --get-passwords

# Backup config
./sos --backup-config

# Cleanup old logs
./sos --cleanup-logs
```

24. VDT (VMware Depot Tool)

```
# Mixed: first line runs from your client; the rest run on the appliance after
SSH.
# Upload to SDDC Manager (run from your client)
ssh vcf@192.168.1.241 "cat > /home/vcf/vdt.zip" < vdt-2.2.7_02-05-2026.zip

# Run on the appliance only:
su -
cd /home/vcf
unzip vdt-2.2.7_02-05-2026.zip
cd vdt-2.2.7_02-05-2026
python vdt.py
# Prompted for administrator@vsphere.local password
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Step 1 (client-side): upload the zip via OpenSSH cat-redirect.
Get-Content -Raw -Encoding Byte 'vdt-2.2.7_02-05-2026.zip' |
    ssh vcf@192.168.1.241 'cat > /home/vcf/vdt.zip'

# Step 2 (appliance-side): SSH in and run the appliance-only commands.
# ssh vcf@192.168.1.241
# su -
```

```
# cd /home/vcf
# unzip vdt-2.2.7_02-05-2026.zip
# cd vdt-2.2.7_02-05-2026
# python vdt.py
```

Note: VDT is NOT pre-installed. Download from Broadcom KB 344917.

25. ovftool — Manual OVA Deployment

Discover vCenter Inventory Path

```
# bash on any client with ovftool installed
ovftool --noSSLVerify "vi://administrator%40vsphere.local:
<pass>@vcenter.lab.local/"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Default install path for ovftool on Windows
$ovftool = 'C:\Program Files\VMware\VMware OVF Tool\ovftool.exe'
& $ovftool --noSSLVerify "vi://administrator%40vsphere.local:
<pass>@vcenter.lab.local/"
```

Deploy OVA

```
# bash on any client with ovftool installed
ovftool --acceptAllEulas --skipManifestCheck --noSSLVerify \
  --diskMode=thin --powerOn \
  --name=<vm-name> \
  --datastore=vcenter-cl01-ds-vsan01 \
  --net:"Network 1"="vcenter-cl01-vds01-pg-vm-mgmt" \
  --prop:vamitimezone=UTC \
  --prop:root_password='<pass>' \
  "/path/to/appliance.ova" \
```

```
"vi://administrator%40vsphere.local:<url-encoded-
pass>@vcenter.lab.local/vcenter-dc01/host/vcenter-cl01/"
```

PowerShell equivalent (run on your Windows workstation):

```
# PowerShell on Windows workstation
# Single-line invocation per the doc note: "Use single-line commands. Backslash
continuation breaks --noSSLVerify."
# In PS, you can either keep it on one line or use backtick continuation. Backtick
form below.
$ovftool = 'C:\Program Files\VMware\VMware OVF Tool\ovftool.exe'
& $ovftool --acceptAllEulas --skipManifestCheck --noSSLVerify `
    --diskMode=thin --powerOn `
    --name=<vm-name> `
    --datastore=vcenter-cl01-ds-vsan01 `
    --net:"Network 1"="vcenter-cl01-vds01-pg-vm-mgmt" `
    --prop:vamitimezone=UTC `
    "--prop:root_password=<pass>" `
    "C:\path\to\appliance.ova" `
    "vi://administrator%40vsphere.local:<url-encoded-
pass>@vcenter.lab.local/vcenter-dc01/host/vcenter-cl01/"
# Note: Per the doc warning, if backtick continuation triggers issues, collapse to
one line.
```

Note: Use single-line commands. Backslash continuation breaks `--noSSLVerify` .

26. Windows Host — DNS & Firewall

DNS Record Management (PowerShell)

```
# Forward record
Add-DnsServerResourceRecordA -Name "vcenter" -ZoneName "lab.local" -IPv4Address
"192.168.1.69"

# Reverse record
Add-DnsServerResourceRecordPtr -Name "69" -ZoneName "1.168.192.in-addr.arpa" -
PtrDomainName "vcenter.lab.local"
```

```
# Verify
nslookup vcenter.lab.local
nslookup 192.168.1.69

# List all records
Get-DnsServerResourceRecord -ZoneName "lab.local"
```

Windows Certificate Utility

```
certutil -dump cert.crt
certutil -verify cert.crt
certutil -hashfile file.zip SHA256
```

27. Diagnostic Python Scripts

Run from Windows workstation. Requires `pip install paramiko`.

Scenario	Script
Overall health dashboard	<code>python quick_status.py</code>
NSX recovering after boot	<code>python nsx_monitor.py</code>
NSX VIP + node connectivity	<code>python nsx_check.py</code>
NSX performance + services	<code>python nsx_diag.py</code>
SDDC Manager vs NSX sync	<code>python sddc_nsx_status.py</code>
Diagnose remediate failure	<code>python check_remediate_error.py</code>
Clear stale DB locks + fix NSX status	<code>python clear_locks.py</code>
Fix stuck IN_PROGRESS tasks	<code>python fix_stuck_tasks.py</code>
Full cascade fix (all-in-one)	<code>python full_remediate_fix.py</code>
Check disconnected accounts	<code>python check_disconnected.py</code>
Update NSX password	<code>python nsx_cred_update.py</code>
NSX CPU overloaded	<code>python nsx_slim.py</code>

Restore NSX services	<code>python nsx_restart_all.py</code>
System clean after fix?	<code>python final_check.py</code>

All scripts are in `C:\VCF-Depot\diagnostic-scripts\`. Requires `pip install paramiko`. See *D diagnostic-Scripts-Cheatsheet.md* for full reference.

28. GRUB Recovery — SDDC Manager

When all accounts are locked (root, vcf, admin):

1. Reboot VM in VMware Workstation
2. Press `e` at GRUB menu
3. Find the `linux` line, append: `init=/bin/bash`
4. Press `Ctrl+X` to boot
5. At bash prompt:

```
# Run on the appliance only (in-VM single-user shell after GRUB edit)
mount -o remount,rw /
faillock --user root --reset
faillock --user vcf --reset
reboot -f
```

Note: `systemctl` commands will not work in `init=/bin/bash` mode (systemd not running). Just reset faillock and reboot.

Document Information

Field	Value
Document Title	VCF 9.0.1 Command Reference
Version	1.2
Last Updated	May 2026
Environment	Dell Precision 7920, VMware Workstation Nested Lab
Lab IPs	SDDC Mgr .241, vCenter .69, NSX VIP .70, NSX Node .71, ESXi .74-.76/.82

Revision History

Version	Date	Notes
1.0	January 2026	Initial release.
1.1	February 2026	Content additions across NSX/SDDC Manager troubleshooting sections.
1.2	May 2026	Option B PowerShell-sibling pass: every bash block now has a paste-ready PowerShell equivalent. Appliance-only blocks (psql, esxcli, vim-cmd, vmon-cli, service-control, vmkfstools, partedUtil, vdq, esxtop, sos, faillock) flagged with "Run on the appliance only" comments. Added callout linking to POWERSHELL-TRANSLATION-REFERENCE.md.

(c) 2026 Virtual Control LLC. All rights reserved.